

## Assignment-9.3

2303A51242

Batch-05

## Task 1: Basic Docstring Generation

```

1
2 def sum_even_odd(numbers):
3     """
4     Docstring for sum_even_odd
5
6     :param numbers: Description
7     """
8     even_sum = 0
9     odd_sum = 0
10    for num in numbers:
11        if num % 2 == 0:
12            even_sum += num
13        else:
14            odd_sum += num
15    return even_sum, odd_sum
16
17 # Example usage:
18 numbers = [1, 2, 3, 4, 5, 6]
19 even_sum, odd_sum = sum_even_odd(numbers)
20 print("Sum of even numbers:", even_sum) # Output: 12
21 print("Sum of odd numbers:", odd_sum)   # Output: 9
22 print(sum_even_odd.__doc__)
23
24
25 Sum the even and odd numbers in a list separately.
26
27 :param numbers: List of integers
28 :return: A tuple containing the sum of even numbers and the sum of odd numbers
29 """

```

Output:

```

> & C:/Users/kamul/AppData/Local/Programs/Python/Python313/python
n.exe "c:/Users/kamul/OneDrive/Desktop/AI ASSISTANT CODING/assignment-9.py"
Sum of even numbers: 12
Sum of odd numbers: 9

Docstring for sum_even_odd

:param numbers: Description

PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING>

```

## Explanation:

Manual documentation is written by developers and provides accurate, detailed, and context-specific explanations, but it is time-consuming and may become outdated. AI-generated documentation is created automatically, making it faster, consistent, and useful for large codebases, though it may lack deep context or developer intent. AI-generated function-level documentation helps users quickly understand what a function does, its inputs, and outputs without reading the full code. It improves readability, saves time, and supports easier learning and maintenance, especially when combined with human review.

## Task 2: Automatic Inline Comments

```
class sru_student:
    # Constructor method to initialize student attributes
    def __init__(self, name, roll_no, hostel_status):
        self.name = name # Student's name
        self.roll_no = roll_no # Student's roll number
        self.hostel_status = hostel_status # Student's hostel status (e.g., 'Resident', 'Day Scholar')
        self.fees_paid = 0 # Initialize fees paid to zero for new students

    # Method to update the fees paid by the student
    def fee_update(self, amount):
        self.fees_paid += amount # Add the given amount to the total fees paid
        print(f"Fee updated for {self.name} (Roll No: {self.roll_no}). Total fees paid: {self.fees_paid}") # Print update message

    # Method to display all details of the student
    def display_details(self):
        print(f"Student Name: {self.name}") # Display student's name
        print(f"Roll Number: {self.roll_no}") # Display student's roll number
        print(f"Hostel Status: {self.hostel_status}") # Display student's hostel status
        print(f"Fees Paid: {self.fees_paid}") # Display total fees paid

# Example usage of the sru_student class:
# Create the first student object
student1 = sru_student("Alice Smith", "SRU101", "Resident")
# Create the second student object
student2 = sru_student("Bob Johnson", "SRU102", "Day Scholar")

# Display details for student1 before fee update
student1.display_details()
# Update fees for student1

# Example usage of the sru_student class:
# Create the first student object
student1 = sru_student("Alice Smith", "SRU101", "Resident")
# Create the second student object
student2 = sru_student("Bob Johnson", "SRU102", "Day Scholar")

# Display details for student1 before fee update
student1.display_details()
# Update fees for student1
student1.fee_update(50000)
# Display details for student1 after fee update
student1.display_details()

# Display details for student2 before fee update
student2.display_details()
# Update fees for student2
student2.fee_update(25000)
# Display details for student2 after fee update
student2.display_details()

...
Student Name: Alice Smith
Roll Number: SRU101
Hostel Status: Resident
Fees Paid: 0
Fee updated for Alice Smith (Roll No: SRU101). Total fees paid: 50000

Student Name: Alice Smith
...
```

## Output:

```
# Display details for student2 after fee update
student2.display_details()

...
Student Name: Alice Smith
Roll Number: SRU101
Hostel Status: Resident
Fees Paid: 0
Fee updated for Alice Smith (Roll No: SRU101). Total fees paid: 50000

Student Name: Alice Smith
Roll Number: SRU101
Hostel Status: Resident
Fees Paid: 50000

Student Name: Bob Johnson
Roll Number: SRU102
Hostel Status: Day Scholar
Fees Paid: 0
Fee updated for Bob Johnson (Roll No: SRU102). Total fees paid: 25000

Student Name: Bob Johnson
Roll Number: SRU102
Hostel Status: Day Scholar
Fees Paid: 25000
```

## Explanation:

AI-generated inline comments are automatically added within the code to explain individual lines or small blocks of logic. These comments help readers quickly understand what each part of the code does, improving readability and reducing the effort needed to interpret complex sections. In comparison, manual comments are usually more precise and reflect the developer's intent and reasoning, while AI-generated comments are faster, consistent, and easier to produce at scale. However, AI comments may be more generic and might not capture special cases or design decisions. Overall, AI-generated inline comments are useful for quick understanding and documentation, especially when reviewed and refined by developers.

## Task 3: Module-Level and Function-Level Documentation

```
calculator_module.py

A lightweight, reusable Python module providing fundamental arithmetic operations.
This module is designed for clarity, efficiency, and ease of integration into
various projects requiring basic mathematical computations.

Functions:
    add(a, b): Returns the sum of two numbers.
    subtract(a, b): Returns the difference between two numbers.
    multiply(a, b): Returns the product of two numbers.
    divide(a, b): Returns the quotient of two numbers, handling division by zero.

Example Usage:
>>> import calculator_module
>>> calculator_module.add(5, 3)
8
>>> calculator_module.divide(10, 2)
5.0

def add(a, b):
    """
    Adds two numbers and returns their sum.

    Parameters
    -----
    a : int or float
        The first number
```

```
    a : int or float
        The first number.
    b : int or float
        The second number.

    Returns
    -----
    int or float
        The sum of `a` and `b`.

    Examples
    -----
    >>> add(5, 3)
    8
    >>> add(-1, 10)
    9
    >>> add(2.5, 3.5)
    6.0
    """
    return a + b

def subtract(a, b):
    """
    Subtracts the second number from the first and returns the difference.

    Parameters
    -----
    a : int or float
        The number from which to subtract.
    b : int or float
```



Returns

-----

int or float

The difference between `a` and `b` ( $a - b$ ).

Examples

-----

```
>>> subtract(10, 4)
```

```
6
```

```
>>> subtract(5, 8)
```

```
-3
```

```
>>> subtract(7.0, 2.5)
```

```
4.5
```

```
"""
```

```
return a - b
```

```
def multiply(a, b):
```

```
    """
```

```
    Multiplies two numbers and returns their product.
```

Parameters

-----

a : int or float

The first number to multiply.

b : int or float

The second number to multiply.

Returns

-----



int or float

The product of `a` and `b`.

Examples

-----

```
>>> multiply(6, 7)
```

```
42
```

```
>>> multiply(-2, 5)
```

```
-10
```

```
>>> multiply(3.0, 1.5)
```

```
4.5
```

```
"""
```

```
return a * b
```

```
def divide(a, b):
```

```
    """
```

```
    Divides the first number by the second and returns the quotient.
```

```
    Handles division by zero by raising a ValueError.
```

Parameters

-----

a : int or float

The dividend.

b : int or float

The divisor.

Returns

-----

```
.....
-----
int or float
    The quotient of `a` and `b` (a / b).

Raises
-----
ValueError
    If `b` (the divisor) is zero.

Examples
-----
>>> divide(10, 2)
5.0
>>> divide(7, 0.5)
14.0
>>> divide(5, 0)
Traceback (most recent call last):
...
ValueError: Cannot divide by zero.
"""

if b == 0:
    raise ValueError("Cannot divide by zero.")
return a / b

# Import the calculator module
# Since the module was defined in the current notebook, its functions are directly available.
# However, if this were a separate file, you would use 'import calculator_module'.

# Perform some calculations using the functions
result_add = add(10, 5)
```

Output:

```
...
    Adds two numbers and returns their sum.

Parameters
-----
a : int or float
    The first number.
b : int or float
    The second number.

Returns
-----
int or float
    The sum of `a` and `b`.

Examples
-----
>>> add(5, 3)
8
>>> add(-1, 10)
9
>>> add(2.5, 3.5)
6.0

    Subtracts the second number from the first and returns the difference.

Parameters
-----
a : int or float
```

```
Parameters
-----
a : int or float
    The number from which to subtract.
b : int or float
    The number to subtract.

Returns
-----
int or float
    The difference between `a` and `b` (a - b).

Examples
-----
>>> subtract(10, 4)
6
>>> subtract(5, 8)
-3
>>> subtract(7.0, 2.5)
4.5

Multiplies two numbers and returns their product.

Parameters
-----
a : int or float
    The first number to multiply.
b : int or float
    The second number to multiply.
```

```
Returns
-----
int or float
    The product of `a` and `b`.

Examples
-----
>>> multiply(6, 7)
42
>>> multiply(-2, 5)
-10
>>> multiply(3.0, 1.5)
4.5

Divides the first number by the second and returns the quotient.

Handles division by zero by raising a ValueError.

Parameters
-----
a : int or float
    The dividend.
b : int or float
    The divisor.

Returns
-----
int or float
    The quotient of `a` and `b` (a / b).
```

```
Raises
-----
ValueError
    If `b` (the divisor) is zero.

Examples
-----
>>> divide(10, 2)
5.0
>>> divide(7, 0.5)
14.0
>>> divide(5, 0)
Traceback (most recent call last):
...
ValueError: Cannot divide by zero.

10 + 5 = 15
10 - 5 = 5
10 * 5 = 50
10 / 5 = 2.0
Error: Cannot divide by zero.
```

**Explanation:** Manual documentation is written by developers and provides detailed, context-aware explanations that reflect the original design decisions, but it requires more time and effort to create and maintain. AI-generated documentation is produced automatically, making it faster, consistent, and helpful for large or frequently changing codebases, though it may be more generic and miss deeper intent. Structured documentation for multi-function scripts organizes information clearly by describing each function's purpose, inputs, outputs, and relationships. This structured approach helps readers understand how different functions work together, improves code readability, and makes maintenance and collaboration easier.